



Building a Pet Management App with React

Section 1: Learn

What is the Pet Management App?

A full-stack web application using React as the frontend and Spring Boot as the backend to manage pet records. It includes forms, file uploads, data persistence, and RESTful communication.

Why Build This?

- **Practical Hands-On:** Learn end-to-end development flow.
- **Component-Based UI:** Build dynamic user experiences with React.
- **Data-Driven Logic:** Use Spring Boot for business logic and database interactions.

Architecture Overview

- **Frontend:** React handles UI, form inputs, and API requests.
- **Backend:** Spring Boot handles REST APIs, validation, file handling, and database interaction.
- **Communication:** Axios (in React) sends HTTP requests to Spring Boot endpoints.

Interesting Anecdote

The modular separation in this project mirrors enterprise-level applications like CRM tools and dashboards used in real businesses, making it excellent training for real-world software development.



Section 2: Practice

Step-by-Step Implementation

1. Set Up React Frontend

```
npx create-react-app pet-app  
cd pet-app  
npm install axios
```

2. Add Form for Adding Pets (App.js)

```
<form onSubmit={handleSubmit}>  
  <input name="name" value={pet.name} onChange={handleChange}  
placeholder="Name" />  
  <input name="age" value={pet.age} onChange={handleChange}  
placeholder="Age" />  
  <input name="breed" value={pet.breed} onChange={handleChange}  
placeholder="Breed" />  
  <button type="submit">Add Pet</button>  
</form>
```

3. Submit Form Data to Backend

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  const formData = new FormData();  
  formData.append('name', pet.name);  
  formData.append('age', pet.age);  
  formData.append('breed', pet.breed);
```



```
formData.append('file', selectedFile);

await axios.post('http://localhost:8080/pet/addpet', formData);
};
```

4. Display Pet List from Backend

```
const fetchPets = async () => {
  const res = await axios.get('http://localhost:8080/pet/fetchdata');
  setPets(res.data);
};

<ul>

  {pets.map((pet, index) => (
    <li key={index}>{pet.name} - {pet.age} years - {pet.breed}</li>

  ))}

</ul>
```

File Upload in Spring Boot

application.yml

```
upload-dir: uploads/
```

Controller for File Upload

```
@PostMapping("/upload")
public ResponseEntity<?> uploadFile(@RequestParam("file") MultipartFile file)
{
  String filename = System.currentTimeMillis() + "_" +
```



```
file.getOriginalFilename();  
    Path path = Paths.get(uploadDir + filename);  
    file.transferTo(path);  
    return ResponseEntity.ok("File uploaded");  
}
```

Controller for File Download

```
@GetMapping("/download/{filename}")  
public ResponseEntity<Resource> downloadFile(@PathVariable String  
filename) {  
    Path file = Paths.get(uploadDir).resolve(filename);  
    Resource resource = new UrlResource(file.toUri());  
    return ResponseEntity.ok()  
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=" +  
filename)  
        .body(resource);  
}
```

React Upload Component

```
<input type="file" onChange={(e) => setSelectedFile(e.target.files[0])} />
```

React Download and Image Display

```
<img src={`http://localhost:8080/pet/download/${pet.image}`} alt="Pet" />  
<a href={`http://localhost:8080/pet/download/${pet.image}`}  
download>Download</a>
```



Section 3: Know More

FAQs

Q1: Why use FormData in React?

- **Answer:** To send both file and form data in one request using `multipart/form-data`.

Q2: How do I avoid CORS issues?

- **Answer:** Configure `@CrossOrigin` in Spring Boot or proxy in React.

Q3: Can uploaded files be stored in the database?

- **Answer:** Typically no. Store files in the file system or cloud and store only metadata in the DB.

Q4: How are images displayed after upload?

- **Answer:** By creating URLs pointing to download endpoints and rendering them using `<img>` tags.

Q5: Can this architecture scale?

- **Answer:** Yes. It's modular and stateless, making it suitable for enterprise deployment.